



VOLUME 03

MATHEMATICS, SCENES, WORLDS AND LARGE COORDINATES

Mathematics, transforms, scene graph, components, spatial indexing, world cells, streaming and precision-safe large worlds.

DOCUMENT CODE	SKEIN-IMP-002-V03
VERSION	0.1
STATUS	DRAFT CONSTRUCTION REFERENCE
PLANNED PAGES	180
CHAPTERS	14
DATE	30 July 2026

IMPLEMENTATION MANUAL / PRINT SET

Current architecture source: SKEIN-SPEC-001 v0.4 and SKEIN-IMP-002-MASTER v0.1

03.01 Mathematics conventions

Purpose and boundary: handedness

This page defines the purpose and boundary for Mathematics conventions, with particular attention to handedness. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Handedness is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with axes is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for handedness; distinguish required behavior from illustrative implementation detail.
- Keep axes behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class MathematicsConventionsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const MathematicsConventionsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    MathematicsConventionsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.01 and handedness.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.01 Mathematics conventions

Architecture: axes

This page defines the architecture for Mathematics conventions, with particular attention to axes. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Axes is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with angles is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for axes; distinguish required behavior from illustrative implementation detail.
- Keep angles behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Mathematics", "MathematicsConventions");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.01 and axes.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.01 Mathematics conventions

Data model: angles

This page defines the data model for Mathematics conventions, with particular attention to angles. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Angles is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with matrix storage is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for angles; distinguish required behavior from illustrative implementation detail.
- Keep matrix storage behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class MathematicsConventionsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const MathematicsConventionsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    MathematicsConventionsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.01 and angles.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.01 Mathematics conventions

Public API: matrix storage

This page defines the public api for Mathematics conventions, with particular attention to matrix storage. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Matrix storage is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with precision rules is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for matrix storage; distinguish required behavior from illustrative implementation detail.
- Keep precision rules behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Mathematics", "MathematicsConventions");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.01 and matrix storage.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.01 Mathematics conventions

Ownership and lifetime: precision rules

This page defines the ownership and lifetime for Mathematics conventions, with particular attention to precision rules. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Precision rules is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with handedness is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for precision rules; distinguish required behavior from illustrative implementation detail.
- Keep handedness behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class MathematicsConventionsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const MathematicsConventionsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    MathematicsConventionsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.01 and precision rules.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.01 Mathematics conventions

Threading model: handedness

This page defines the threading model for Mathematics conventions, with particular attention to handedness. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Handedness is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with axes is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for handedness; distinguish required behavior from illustrative implementation detail.
- Keep axes behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Mathematics", "MathematicsConventions");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.01 and handedness.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.01 Mathematics conventions

Memory strategy: axes

This page defines the memory strategy for Mathematics conventions, with particular attention to axes. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Axes is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with angles is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for axes; distinguish required behavior from illustrative implementation detail.
- Keep angles behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class MathematicsConventionsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const MathematicsConventionsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    MathematicsConventionsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.01 and axes.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.01 Mathematics conventions

Failure handling: angles

This page defines the failure handling for Mathematics conventions, with particular attention to angles. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Angles is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with matrix storage is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for angles; distinguish required behavior from illustrative implementation detail.
- Keep matrix storage behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Mathematics", "MathematicsConventions");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.01 and angles.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.01 Mathematics conventions

Diagnostics: matrix storage

This page defines the diagnostics for Mathematics conventions, with particular attention to matrix storage. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Matrix storage is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with precision rules is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for matrix storage; distinguish required behavior from illustrative implementation detail.
- Keep precision rules behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class MathematicsConventionsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const MathematicsConventionsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    MathematicsConventionsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.01 and matrix storage.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.01 Mathematics conventions

Implementation sequence: precision rules

This page defines the implementation sequence for Mathematics conventions, with particular attention to precision rules. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Precision rules is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with handedness is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for precision rules; distinguish required behavior from illustrative implementation detail.
- Keep handedness behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Mathematics", "MathematicsConventions");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.01 and precision rules.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.01 Mathematics conventions

Verification: handedness

This page defines the verification for Mathematics conventions, with particular attention to handedness. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Handedness is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with axes is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for handedness; distinguish required behavior from illustrative implementation detail.
- Keep axes behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class MathematicsConventionsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const MathematicsConventionsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    MathematicsConventionsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.01 and handedness.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.02 Vectors, matrices and quaternions

Purpose and boundary: SIMD boundary

This page defines the purpose and boundary for Vectors, matrices and quaternions, with particular attention to SIMD boundary. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Simd boundary is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with normalization is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for SIMD boundary; distinguish required behavior from illustrative implementation detail.
- Keep normalization behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VectorsMatricesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VectorsMatricesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VectorsMatricesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.02 and SIMD boundary.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.02 Vectors, matrices and quaternions

Architecture: normalization

This page defines the architecture for Vectors, matrices and quaternions, with particular attention to normalization. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Normalization is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with robust comparisons is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for normalization; distinguish required behavior from illustrative implementation detail.
- Keep robust comparisons behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vectors", "VectorsMatricesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.02 and normalization.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.02 Vectors, matrices and quaternions

Data model: robust comparisons

This page defines the data model for Vectors, matrices and quaternions, with particular attention to robust comparisons. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Robust comparisons is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tests is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for robust comparisons; distinguish required behavior from illustrative implementation detail.
- Keep tests behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VectorsMatricesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VectorsMatricesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VectorsMatricesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.02 and robust comparisons.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.02 Vectors, matrices and quaternions

Public API: tests

This page defines the public api for Vectors, matrices and quaternions, with particular attention to tests. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Tests is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with SIMD boundary is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tests; distinguish required behavior from illustrative implementation detail.
- Keep SIMD boundary behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vectors", "VectorsMatricesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.02 and tests.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.02 Vectors, matrices and quaternions

Ownership and lifetime: SIMD boundary

This page defines the ownership and lifetime for Vectors, matrices and quaternions, with particular attention to SIMD boundary. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Simd boundary is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with normalization is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for SIMD boundary; distinguish required behavior from illustrative implementation detail.
- Keep normalization behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VectorsMatricesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VectorsMatricesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VectorsMatricesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.02 and SIMD boundary.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.02 Vectors, matrices and quaternions

Threading model: normalization

This page defines the threading model for Vectors, matrices and quaternions, with particular attention to normalization. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Normalization is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with robust comparisons is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for normalization; distinguish required behavior from illustrative implementation detail.
- Keep robust comparisons behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vectors", "VectorsMatricesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.02 and normalization.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.02 Vectors, matrices and quaternions

Memory strategy: robust comparisons

This page defines the memory strategy for Vectors, matrices and quaternions, with particular attention to robust comparisons. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Robust comparisons is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tests is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for robust comparisons; distinguish required behavior from illustrative implementation detail.
- Keep tests behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VectorsMatricesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VectorsMatricesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VectorsMatricesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.02 and robust comparisons.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.02 Vectors, matrices and quaternions

Failure handling: tests

This page defines the failure handling for Vectors, matrices and quaternions, with particular attention to tests. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Tests is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with SIMD boundary is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tests; distinguish required behavior from illustrative implementation detail.
- Keep SIMD boundary behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vectors", "VectorsMatricesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.02 and tests.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.02 Vectors, matrices and quaternions

Diagnostics: SIMD boundary

This page defines the diagnostics for Vectors, matrices and quaternions, with particular attention to SIMD boundary. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Simd boundary is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with normalization is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for SIMD boundary; distinguish required behavior from illustrative implementation detail.
- Keep normalization behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VectorsMatricesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VectorsMatricesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VectorsMatricesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.02 and SIMD boundary.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.02 Vectors, matrices and quaternions

Implementation sequence: normalization

This page defines the implementation sequence for Vectors, matrices and quaternions, with particular attention to normalization. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Normalization is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with robust comparisons is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for normalization; distinguish required behavior from illustrative implementation detail.
- Keep robust comparisons behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vectors", "VectorsMatricesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.02 and normalization.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.02 Vectors, matrices and quaternions

Verification: robust comparisons

This page defines the verification for Vectors, matrices and quaternions, with particular attention to robust comparisons. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Robust comparisons is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tests is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for robust comparisons; distinguish required behavior from illustrative implementation detail.
- Keep tests behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VectorsMatricesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VectorsMatricesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VectorsMatricesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.02 and robust comparisons.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.02 Vectors, matrices and quaternions

Completion gate: tests

This page defines the completion gate for Vectors, matrices and quaternions, with particular attention to tests. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Tests is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with SIMD boundary is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tests; distinguish required behavior from illustrative implementation detail.
- Keep SIMD boundary behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vectors", "VectorsMatricesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.02 and tests.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.02 Vectors, matrices and quaternions

Purpose and boundary: SIMD boundary

This page defines the purpose and boundary for Vectors, matrices and quaternions, with particular attention to SIMD boundary. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Simd boundary is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with normalization is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for SIMD boundary; distinguish required behavior from illustrative implementation detail.
- Keep normalization behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class VectorsMatricesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const VectorsMatricesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    VectorsMatricesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.02 and SIMD boundary.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.02 Vectors, matrices and quaternions

Architecture: normalization

This page defines the architecture for Vectors, matrices and quaternions, with particular attention to normalization. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Normalization is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with robust comparisons is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for normalization; distinguish required behavior from illustrative implementation detail.
- Keep robust comparisons behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Vectors", "VectorsMatricesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.02 and normalization.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.03 Transforms and hierarchy propagation

Purpose and boundary: local/world transforms

This page defines the purpose and boundary for Transforms and hierarchy propagation, with particular attention to local/world transforms. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Local/world transforms is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with dirty propagation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for local/world transforms; distinguish required behavior from illustrative implementation detail.
- Keep dirty propagation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class TransformsAndHierarchyService final
{
public:
    [[nodiscard]] Result<void> Initialise(const TransformsAndHierarchyConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    TransformsAndHierarchyState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.03 and local/world transforms.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.03 Transforms and hierarchy propagation

Architecture: dirty propagation

This page defines the architecture for Transforms and hierarchy propagation, with particular attention to dirty propagation. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Dirty propagation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cycles is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for dirty propagation; distinguish required behavior from illustrative implementation detail.
- Keep cycles behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Transforms", "TransformsAndHierarchy");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.03 and dirty propagation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.03 Transforms and hierarchy propagation

Data model: cycles

This page defines the data model for Transforms and hierarchy propagation, with particular attention to cycles. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Cycles is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with batch updates is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cycles; distinguish required behavior from illustrative implementation detail.
- Keep batch updates behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class TransformsAndHierarchyService final
{
public:
    [[nodiscard]] Result<void> Initialise(const TransformsAndHierarchyConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    TransformsAndHierarchyState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.03 and cycles.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.03 Transforms and hierarchy propagation

Public API: batch updates

This page defines the public api for Transforms and hierarchy propagation, with particular attention to batch updates. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Batch updates is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with local/world transforms is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for batch updates; distinguish required behavior from illustrative implementation detail.
- Keep local/world transforms behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Transforms", "TransformsAndHierarchy");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.03 and batch updates.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.03 Transforms and hierarchy propagation

Ownership and lifetime: local/world transforms

This page defines the ownership and lifetime for Transforms and hierarchy propagation, with particular attention to local/world transforms. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Local/world transforms is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with dirty propagation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for local/world transforms; distinguish required behavior from illustrative implementation detail.
- Keep dirty propagation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class TransformsAndHierarchyService final
{
public:
    [[nodiscard]] Result<void> Initialise(const TransformsAndHierarchyConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    TransformsAndHierarchyState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.03 and local/world transforms.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.03 Transforms and hierarchy propagation

Threading model: dirty propagation

This page defines the threading model for Transforms and hierarchy propagation, with particular attention to dirty propagation. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Dirty propagation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cycles is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for dirty propagation; distinguish required behavior from illustrative implementation detail.
- Keep cycles behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Transforms", "TransformsAndHierarchy");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.03 and dirty propagation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.03 Transforms and hierarchy propagation

Memory strategy: cycles

This page defines the memory strategy for Transforms and hierarchy propagation, with particular attention to cycles. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Cycles is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with batch updates is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cycles; distinguish required behavior from illustrative implementation detail.
- Keep batch updates behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class TransformsAndHierarchyService final
{
public:
    [[nodiscard]] Result<void> Initialise(const TransformsAndHierarchyConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    TransformsAndHierarchyState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.03 and cycles.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.03 Transforms and hierarchy propagation

Failure handling: batch updates

This page defines the failure handling for Transforms and hierarchy propagation, with particular attention to batch updates. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Batch updates is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with local/world transforms is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for batch updates; distinguish required behavior from illustrative implementation detail.
- Keep local/world transforms behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Transforms", "TransformsAndHierarchy");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.03 and batch updates.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.03 Transforms and hierarchy propagation

Diagnostics: local/world transforms

This page defines the diagnostics for Transforms and hierarchy propagation, with particular attention to local/world transforms. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Local/world transforms is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with dirty propagation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for local/world transforms; distinguish required behavior from illustrative implementation detail.
- Keep dirty propagation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class TransformsAndHierarchyService final
{
public:
    [[nodiscard]] Result<void> Initialise(const TransformsAndHierarchyConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    TransformsAndHierarchyState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.03 and local/world transforms.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.03 Transforms and hierarchy propagation

Implementation sequence: dirty propagation

This page defines the implementation sequence for Transforms and hierarchy propagation, with particular attention to dirty propagation. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Dirty propagation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cycles is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for dirty propagation; distinguish required behavior from illustrative implementation detail.
- Keep cycles behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Transforms", "TransformsAndHierarchy");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.03 and dirty propagation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.03 Transforms and hierarchy propagation

Verification: cycles

This page defines the verification for Transforms and hierarchy propagation, with particular attention to cycles. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Cycles is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with batch updates is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cycles; distinguish required behavior from illustrative implementation detail.
- Keep batch updates behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class TransformsAndHierarchyService final
{
public:
    [[nodiscard]] Result<void> Initialise(const TransformsAndHierarchyConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    TransformsAndHierarchyState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.03 and cycles.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.03 Transforms and hierarchy propagation

Completion gate: batch updates

This page defines the completion gate for Transforms and hierarchy propagation, with particular attention to batch updates. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Batch updates is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with local/world transforms is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for batch updates; distinguish required behavior from illustrative implementation detail.
- Keep local/world transforms behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Transforms", "TransformsAndHierarchy");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.03 and batch updates.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.03 Transforms and hierarchy propagation

Purpose and boundary: local/world transforms

This page defines the purpose and boundary for Transforms and hierarchy propagation, with particular attention to local/world transforms. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Local/world transforms is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with dirty propagation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for local/world transforms; distinguish required behavior from illustrative implementation detail.
- Keep dirty propagation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class TransformsAndHierarchyService final
{
public:
    [[nodiscard]] Result<void> Initialise(const TransformsAndHierarchyConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    TransformsAndHierarchyState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.03 and local/world transforms.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.03 Transforms and hierarchy propagation

Architecture: dirty propagation

This page defines the architecture for Transforms and hierarchy propagation, with particular attention to dirty propagation. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Dirty propagation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cycles is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for dirty propagation; distinguish required behavior from illustrative implementation detail.
- Keep cycles behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Transforms", "TransformsAndHierarchy");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.03 and dirty propagation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.03 Transforms and hierarchy propagation

Data model: cycles

This page defines the data model for Transforms and hierarchy propagation, with particular attention to cycles. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Cycles is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with batch updates is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cycles; distinguish required behavior from illustrative implementation detail.
- Keep batch updates behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class TransformsAndHierarchyService final
{
public:
    [[nodiscard]] Result<void> Initialise(const TransformsAndHierarchyConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    TransformsAndHierarchyState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.03 and cycles.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.03 Transforms and hierarchy propagation

Public API: batch updates

This page defines the public api for Transforms and hierarchy propagation, with particular attention to batch updates. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Batch updates is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with local/world transforms is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for batch updates; distinguish required behavior from illustrative implementation detail.
- Keep local/world transforms behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Transforms", "TransformsAndHierarchy");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.03 and batch updates.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.04 Bounds and geometric primitives

Purpose and boundary: AABB

This page defines the purpose and boundary for Bounds and geometric primitives, with particular attention to AABB. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Aabb is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with OBB is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for AABB; distinguish required behavior from illustrative implementation detail.
- Keep OBB behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class BoundsAndGeometricService final
{
public:
    [[nodiscard]] Result<void> Initialise(const BoundsAndGeometricConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    BoundsAndGeometricState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.04 and AABB.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.04 Bounds and geometric primitives

Architecture: OBB

This page defines the architecture for Bounds and geometric primitives, with particular attention to OBB. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

OBB is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with spheres is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for OBB; distinguish required behavior from illustrative implementation detail.
- Keep spheres behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Bounds", "BoundsAndGeometric");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.04 and OBB.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.04 Bounds and geometric primitives

Data model: spheres

This page defines the data model for Bounds and geometric primitives, with particular attention to spheres. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Spheres is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with planes is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for spheres; distinguish required behavior from illustrative implementation detail.
- Keep planes behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class BoundsAndGeometricService final
{
public:
    [[nodiscard]] Result<void> Initialise(const BoundsAndGeometricConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    BoundsAndGeometricState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.04 and spheres.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.04 Bounds and geometric primitives

Public API: planes

This page defines the public api for Bounds and geometric primitives, with particular attention to planes. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Planes is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with frusta is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for planes; distinguish required behavior from illustrative implementation detail.
- Keep frusta behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Bounds", "BoundsAndGeometric");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.04 and planes.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.04 Bounds and geometric primitives

Ownership and lifetime: frusta

This page defines the ownership and lifetime for Bounds and geometric primitives, with particular attention to frusta. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Frusta is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with rays is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for frusta; distinguish required behavior from illustrative implementation detail.
- Keep rays behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class BoundsAndGeometricService final
{
public:
    [[nodiscard]] Result<void> Initialise(const BoundsAndGeometricConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    BoundsAndGeometricState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.04 and frusta.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.04 Bounds and geometric primitives

Threading model: rays

This page defines the threading model for Bounds and geometric primitives, with particular attention to rays. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Rays is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with AABB is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for rays; distinguish required behavior from illustrative implementation detail.
- Keep AABB behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Bounds", "BoundsAndGeometric");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.04 and rays.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.04 Bounds and geometric primitives

Memory strategy: AABB

This page defines the memory strategy for Bounds and geometric primitives, with particular attention to AABB. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Aabb is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with OBB is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for AABB; distinguish required behavior from illustrative implementation detail.
- Keep OBB behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class BoundsAndGeometricService final
{
public:
    [[nodiscard]] Result<void> Initialise(const BoundsAndGeometricConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    BoundsAndGeometricState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.04 and AABB.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.04 Bounds and geometric primitives

Failure handling: OBB

This page defines the failure handling for Bounds and geometric primitives, with particular attention to OBB. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

OBB is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with spheres is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for OBB; distinguish required behavior from illustrative implementation detail.
- Keep spheres behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Bounds", "BoundsAndGeometric");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.04 and OBB.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.04 Bounds and geometric primitives

Diagnostics: spheres

This page defines the diagnostics for Bounds and geometric primitives, with particular attention to spheres. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Spheres is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with planes is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for spheres; distinguish required behavior from illustrative implementation detail.
- Keep planes behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class BoundsAndGeometricService final
{
public:
    [[nodiscard]] Result<void> Initialise(const BoundsAndGeometricConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    BoundsAndGeometricState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.04 and spheres.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.04 Bounds and geometric primitives

Implementation sequence: planes

This page defines the implementation sequence for Bounds and geometric primitives, with particular attention to planes. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Planes is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with frusta is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for planes; distinguish required behavior from illustrative implementation detail.
- Keep frusta behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Bounds", "BoundsAndGeometric");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.04 and planes.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.04 Bounds and geometric primitives

Verification: frusta

This page defines the verification for Bounds and geometric primitives, with particular attention to frusta. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Frusta is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with rays is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for frusta; distinguish required behavior from illustrative implementation detail.
- Keep rays behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class BoundsAndGeometricService final
{
public:
    [[nodiscard]] Result<void> Initialise(const BoundsAndGeometricConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    BoundsAndGeometricState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.04 and frusta.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.05 Double-precision world coordinates

Purpose and boundary: WorldPosition

This page defines the purpose and boundary for Double-precision world coordinates, with particular attention to WorldPosition. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Worldposition is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cell+offset is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for WorldPosition; distinguish required behavior from illustrative implementation detail.
- Keep cell+offset behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DoublePrecisionWorldService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DoublePrecisionWorldConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DoublePrecisionWorldState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.05 and WorldPosition.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.05 Double-precision world coordinates

Architecture: cell+offset

This page defines the architecture for Double-precision world coordinates, with particular attention to cell+offset. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Cell+offset is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with serialization is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cell+offset; distinguish required behavior from illustrative implementation detail.
- Keep serialization behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Double", "DoublePrecisionWorld");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.05 and cell+offset.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.05 Double-precision world coordinates

Data model: serialization

This page defines the data model for Double-precision world coordinates, with particular attention to serialization. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Serialization is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with origin-independent math is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for serialization; distinguish required behavior from illustrative implementation detail.
- Keep origin-independent math behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DoublePrecisionWorldService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DoublePrecisionWorldConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DoublePrecisionWorldState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.05 and serialization.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.05 Double-precision world coordinates

Public API: origin-independent math

This page defines the public api for Double-precision world coordinates, with particular attention to origin-independent math. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Origin-independent math is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with WorldPosition is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for origin-independent math; distinguish required behavior from illustrative implementation detail.
- Keep WorldPosition behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Double", "DoublePrecisionWorld");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.05 and origin-independent math.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.05 Double-precision world coordinates

Ownership and lifetime: WorldPosition

This page defines the ownership and lifetime for Double-precision world coordinates, with particular attention to WorldPosition. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Worldposition is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cell+offset is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for WorldPosition; distinguish required behavior from illustrative implementation detail.
- Keep cell+offset behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DoublePrecisionWorldService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DoublePrecisionWorldConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DoublePrecisionWorldState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.05 and WorldPosition.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.05 Double-precision world coordinates

Threading model: cell+offset

This page defines the threading model for Double-precision world coordinates, with particular attention to cell+offset. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Cell+offset is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with serialization is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cell+offset; distinguish required behavior from illustrative implementation detail.
- Keep serialization behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Double", "DoublePrecisionWorld");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.05 and cell+offset.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.05 Double-precision world coordinates

Memory strategy: serialization

This page defines the memory strategy for Double-precision world coordinates, with particular attention to serialization. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Serialization is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with origin-independent math is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for serialization; distinguish required behavior from illustrative implementation detail.
- Keep origin-independent math behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DoublePrecisionWorldService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DoublePrecisionWorldConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DoublePrecisionWorldState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.05 and serialization.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.05 Double-precision world coordinates

Failure handling: origin-independent math

This page defines the failure handling for Double-precision world coordinates, with particular attention to origin-independent math. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Origin-independent math is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with `WorldPosition` is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and `SkeinTrace` markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for origin-independent math; distinguish required behavior from illustrative implementation detail.
- Keep `WorldPosition` behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Double", "DoublePrecisionWorld");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.05 and origin-independent math.
- `SkeinTrace` events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for `msvc-debug`, `msvc-release`, `clang-debug`, and `clang-release`.

03.05 Double-precision world coordinates

Diagnostics: WorldPosition

This page defines the diagnostics for Double-precision world coordinates, with particular attention to WorldPosition. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Worldposition is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cell+offset is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for WorldPosition; distinguish required behavior from illustrative implementation detail.
- Keep cell+offset behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DoublePrecisionWorldService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DoublePrecisionWorldConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DoublePrecisionWorldState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.05 and WorldPosition.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.05 Double-precision world coordinates

Implementation sequence: cell+offset

This page defines the implementation sequence for Double-precision world coordinates, with particular attention to cell+offset. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Cell+offset is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with serialization is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cell+offset; distinguish required behavior from illustrative implementation detail.
- Keep serialization behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Double", "DoublePrecisionWorld");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.05 and cell+offset.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.05 Double-precision world coordinates

Verification: serialization

This page defines the verification for Double-precision world coordinates, with particular attention to serialization. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Serialization is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with origin-independent math is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for serialization; distinguish required behavior from illustrative implementation detail.
- Keep origin-independent math behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DoublePrecisionWorldService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DoublePrecisionWorldConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DoublePrecisionWorldState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.05 and serialization.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.05 Double-precision world coordinates

Completion gate: origin-independent math

This page defines the completion gate for Double-precision world coordinates, with particular attention to origin-independent math. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Origin-independent math is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with `WorldPosition` is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and `SkeinTrace` markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for origin-independent math; distinguish required behavior from illustrative implementation detail.
- Keep `WorldPosition` behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Double", "DoublePrecisionWorld");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.05 and origin-independent math.
- `SkeinTrace` events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for `msvc-debug`, `msvc-release`, `clang-debug`, and `clang-release`.

03.05 Double-precision world coordinates

Purpose and boundary: WorldPosition

This page defines the purpose and boundary for Double-precision world coordinates, with particular attention to WorldPosition. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Worldposition is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cell+offset is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for WorldPosition; distinguish required behavior from illustrative implementation detail.
- Keep cell+offset behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DoublePrecisionWorldService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DoublePrecisionWorldConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DoublePrecisionWorldState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.05 and WorldPosition.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.05 Double-precision world coordinates

Architecture: cell+offset

This page defines the architecture for Double-precision world coordinates, with particular attention to cell+offset. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Cell+offset is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with serialization is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cell+offset; distinguish required behavior from illustrative implementation detail.
- Keep serialization behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Double", "DoublePrecisionWorld");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.05 and cell+offset.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.05 Double-precision world coordinates

Data model: serialization

This page defines the data model for Double-precision world coordinates, with particular attention to serialization. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Serialization is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with origin-independent math is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for serialization; distinguish required behavior from illustrative implementation detail.
- Keep origin-independent math behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class DoublePrecisionWorldService final
{
public:
    [[nodiscard]] Result<void> Initialise(const DoublePrecisionWorldConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    DoublePrecisionWorldState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.05 and serialization.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.05 Double-precision world coordinates

Public API: origin-independent math

This page defines the public api for Double-precision world coordinates, with particular attention to origin-independent math. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Origin-independent math is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with WorldPosition is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for origin-independent math; distinguish required behavior from illustrative implementation detail.
- Keep WorldPosition behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Double", "DoublePrecisionWorld");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.05 and origin-independent math.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.06 Camera-relative render space

Purpose and boundary: conversion contracts

This page defines the purpose and boundary for Camera-relative render space, with particular attention to conversion contracts. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Conversion contracts is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with precision budgets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for conversion contracts; distinguish required behavior from illustrative implementation detail.
- Keep precision budgets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CameraRelativeRenderService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CameraRelativeRenderConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CameraRelativeRenderState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.06 and conversion contracts.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.06 Camera-relative render space

Architecture: precision budgets

This page defines the architecture for Camera-relative render space, with particular attention to precision budgets. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Precision budgets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with GPU matrices is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for precision budgets; distinguish required behavior from illustrative implementation detail.
- Keep GPU matrices behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Camera", "CameraRelativeRender");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.06 and precision budgets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.06 Camera-relative render space

Data model: GPU matrices

This page defines the data model for Camera-relative render space, with particular attention to GPU matrices. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Gpu matrices is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with debugging is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for GPU matrices; distinguish required behavior from illustrative implementation detail.
- Keep debugging behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CameraRelativeRenderService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CameraRelativeRenderConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CameraRelativeRenderState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.06 and GPU matrices.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.06 Camera-relative render space

Public API: debugging

This page defines the public api for Camera-relative render space, with particular attention to debugging. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Debugging is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with conversion contracts is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for debugging; distinguish required behavior from illustrative implementation detail.
- Keep conversion contracts behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Camera", "CameraRelativeRender");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.06 and debugging.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.06 Camera-relative render space

Ownership and lifetime: conversion contracts

This page defines the ownership and lifetime for Camera-relative render space, with particular attention to conversion contracts. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Conversion contracts is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with precision budgets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for conversion contracts; distinguish required behavior from illustrative implementation detail.
- Keep precision budgets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CameraRelativeRenderService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CameraRelativeRenderConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CameraRelativeRenderState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.06 and conversion contracts.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.06 Camera-relative render space

Threading model: precision budgets

This page defines the threading model for Camera-relative render space, with particular attention to precision budgets. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Precision budgets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with GPU matrices is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for precision budgets; distinguish required behavior from illustrative implementation detail.
- Keep GPU matrices behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Camera", "CameraRelativeRender");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.06 and precision budgets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.06 Camera-relative render space

Memory strategy: GPU matrices

This page defines the memory strategy for Camera-relative render space, with particular attention to GPU matrices. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Gpu matrices is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with debugging is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for GPU matrices; distinguish required behavior from illustrative implementation detail.
- Keep debugging behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CameraRelativeRenderService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CameraRelativeRenderConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CameraRelativeRenderState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.06 and GPU matrices.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.06 Camera-relative render space

Failure handling: debugging

This page defines the failure handling for Camera-relative render space, with particular attention to debugging. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Debugging is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with conversion contracts is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for debugging; distinguish required behavior from illustrative implementation detail.
- Keep conversion contracts behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Camera", "CameraRelativeRender");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.06 and debugging.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.06 Camera-relative render space

Diagnostics: conversion contracts

This page defines the diagnostics for Camera-relative render space, with particular attention to conversion contracts. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Conversion contracts is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with precision budgets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for conversion contracts; distinguish required behavior from illustrative implementation detail.
- Keep precision budgets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CameraRelativeRenderService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CameraRelativeRenderConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CameraRelativeRenderState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.06 and conversion contracts.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.06 Camera-relative render space

Implementation sequence: precision budgets

This page defines the implementation sequence for Camera-relative render space, with particular attention to precision budgets. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Precision budgets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with GPU matrices is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for precision budgets; distinguish required behavior from illustrative implementation detail.
- Keep GPU matrices behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Camera", "CameraRelativeRender");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.06 and precision budgets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.06 Camera-relative render space

Verification: GPU matrices

This page defines the verification for Camera-relative render space, with particular attention to GPU matrices. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Gpu matrices is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with debugging is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for GPU matrices; distinguish required behavior from illustrative implementation detail.
- Keep debugging behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CameraRelativeRenderService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CameraRelativeRenderConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CameraRelativeRenderState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.06 and GPU matrices.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.06 Camera-relative render space

Completion gate: debugging

This page defines the completion gate for Camera-relative render space, with particular attention to debugging. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Debugging is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with conversion contracts is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for debugging; distinguish required behavior from illustrative implementation detail.
- Keep conversion contracts behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Camera", "CameraRelativeRender");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.06 and debugging.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.06 Camera-relative render space

Purpose and boundary: conversion contracts

This page defines the purpose and boundary for Camera-relative render space, with particular attention to conversion contracts. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Conversion contracts is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with precision budgets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for conversion contracts; distinguish required behavior from illustrative implementation detail.
- Keep precision budgets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CameraRelativeRenderService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CameraRelativeRenderConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CameraRelativeRenderState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.06 and conversion contracts.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.07 Scene object and component model

Purpose and boundary: UUIDs

This page defines the purpose and boundary for Scene object and component model, with particular attention to UUIDs. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Uuids is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with component storage is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for UUIDs; distinguish required behavior from illustrative implementation detail.
- Keep component storage behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SceneObjectAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SceneObjectAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SceneObjectAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.07 and UUIDs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.07 Scene object and component model

Architecture: component storage

This page defines the architecture for Scene object and component model, with particular attention to component storage. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Component storage is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with activation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for component storage; distinguish required behavior from illustrative implementation detail.
- Keep activation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Scene", "SceneObjectAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.07 and component storage.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.07 Scene object and component model

Data model: activation

This page defines the data model for Scene object and component model, with particular attention to activation. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Activation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tags is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for activation; distinguish required behavior from illustrative implementation detail.
- Keep tags behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SceneObjectAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SceneObjectAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SceneObjectAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.07 and activation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.07 Scene object and component model

Public API: tags

This page defines the public api for Scene object and component model, with particular attention to tags. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Tags is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with layers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tags; distinguish required behavior from illustrative implementation detail.
- Keep layers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Scene", "SceneObjectAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.07 and tags.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.07 Scene object and component model

Ownership and lifetime: layers

This page defines the ownership and lifetime for Scene object and component model, with particular attention to layers. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Layers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with events is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for layers; distinguish required behavior from illustrative implementation detail.
- Keep events behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SceneObjectAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SceneObjectAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SceneObjectAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.07 and layers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.07 Scene object and component model

Threading model: events

This page defines the threading model for Scene object and component model, with particular attention to events. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Events is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with UUIDs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for events; distinguish required behavior from illustrative implementation detail.
- Keep UUIDs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Scene", "SceneObjectAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.07 and events.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.07 Scene object and component model

Memory strategy: UUIDs

This page defines the memory strategy for Scene object and component model, with particular attention to UUIDs. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Uuids is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with component storage is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for UUIDs; distinguish required behavior from illustrative implementation detail.
- Keep component storage behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SceneObjectAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SceneObjectAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SceneObjectAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.07 and UUIDs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.07 Scene object and component model

Failure handling: component storage

This page defines the failure handling for Scene object and component model, with particular attention to component storage. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Component storage is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with activation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for component storage; distinguish required behavior from illustrative implementation detail.
- Keep activation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Scene", "SceneObjectAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.07 and component storage.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.07 Scene object and component model

Diagnostics: activation

This page defines the diagnostics for Scene object and component model, with particular attention to activation. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Activation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tags is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for activation; distinguish required behavior from illustrative implementation detail.
- Keep tags behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SceneObjectAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SceneObjectAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SceneObjectAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.07 and activation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.07 Scene object and component model

Implementation sequence: tags

This page defines the implementation sequence for Scene object and component model, with particular attention to tags. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Tags is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with layers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tags; distinguish required behavior from illustrative implementation detail.
- Keep layers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Scene", "SceneObjectAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.07 and tags.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.07 Scene object and component model

Verification: layers

This page defines the verification for Scene object and component model, with particular attention to layers. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Layers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with events is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for layers; distinguish required behavior from illustrative implementation detail.
- Keep events behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SceneObjectAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SceneObjectAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SceneObjectAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.07 and layers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.07 Scene object and component model

Completion gate: events

This page defines the completion gate for Scene object and component model, with particular attention to events. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Events is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with UUIDs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for events; distinguish required behavior from illustrative implementation detail.
- Keep UUIDs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Scene", "SceneObjectAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.07 and events.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.07 Scene object and component model

Purpose and boundary: UUIDs

This page defines the purpose and boundary for Scene object and component model, with particular attention to UUIDs. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Uuids is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with component storage is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for UUIDs; distinguish required behavior from illustrative implementation detail.
- Keep component storage behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SceneObjectAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SceneObjectAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SceneObjectAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.07 and UUIDs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.07 Scene object and component model

Architecture: component storage

This page defines the architecture for Scene object and component model, with particular attention to component storage. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Component storage is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with activation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for component storage; distinguish required behavior from illustrative implementation detail.
- Keep activation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Scene", "SceneObjectAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.07 and component storage.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.07 Scene object and component model

Data model: activation

This page defines the data model for Scene object and component model, with particular attention to activation. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Activation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tags is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for activation; distinguish required behavior from illustrative implementation detail.
- Keep tags behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SceneObjectAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SceneObjectAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SceneObjectAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.07 and activation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.07 Scene object and component model

Public API: tags

This page defines the public api for Scene object and component model, with particular attention to tags. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Tags is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with layers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tags; distinguish required behavior from illustrative implementation detail.
- Keep layers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Scene", "SceneObjectAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.07 and tags.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.07 Scene object and component model

Ownership and lifetime: layers

This page defines the ownership and lifetime for Scene object and component model, with particular attention to layers. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Layers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with events is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for layers; distinguish required behavior from illustrative implementation detail.
- Keep events behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SceneObjectAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SceneObjectAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SceneObjectAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.07 and layers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.07 Scene object and component model

Threading model: events

This page defines the threading model for Scene object and component model, with particular attention to events. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Events is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with UUIDs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for events; distinguish required behavior from illustrative implementation detail.
- Keep UUIDs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Scene", "SceneObjectAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.07 and events.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.08 Scene loading and instantiation

Purpose and boundary: staging

This page defines the purpose and boundary for Scene loading and instantiation, with particular attention to staging. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Staging is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with reference resolution is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for staging; distinguish required behavior from illustrative implementation detail.
- Keep reference resolution behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SceneLoadingAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SceneLoadingAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SceneLoadingAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.08 and staging.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.08 Scene loading and instantiation

Architecture: reference resolution

This page defines the architecture for Scene loading and instantiation, with particular attention to reference resolution. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Reference resolution is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with activation phases is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for reference resolution; distinguish required behavior from illustrative implementation detail.
- Keep activation phases behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Scene", "SceneLoadingAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.08 and reference resolution.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.08 Scene loading and instantiation

Data model: activation phases

This page defines the data model for Scene loading and instantiation, with particular attention to activation phases. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Activation phases is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with failure rollback is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for activation phases; distinguish required behavior from illustrative implementation detail.
- Keep failure rollback behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SceneLoadingAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SceneLoadingAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SceneLoadingAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.08 and activation phases.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.08 Scene loading and instantiation

Public API: failure rollback

This page defines the public api for Scene loading and instantiation, with particular attention to failure rollback. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Failure rollback is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with staging is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for failure rollback; distinguish required behavior from illustrative implementation detail.
- Keep staging behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Scene", "SceneLoadingAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.08 and failure rollback.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.08 Scene loading and instantiation

Ownership and lifetime: staging

This page defines the ownership and lifetime for Scene loading and instantiation, with particular attention to staging. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Staging is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with reference resolution is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for staging; distinguish required behavior from illustrative implementation detail.
- Keep reference resolution behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SceneLoadingAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SceneLoadingAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SceneLoadingAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.08 and staging.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.08 Scene loading and instantiation

Threading model: reference resolution

This page defines the threading model for Scene loading and instantiation, with particular attention to reference resolution. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Reference resolution is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with activation phases is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for reference resolution; distinguish required behavior from illustrative implementation detail.
- Keep activation phases behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Scene", "SceneLoadingAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.08 and reference resolution.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.08 Scene loading and instantiation

Memory strategy: activation phases

This page defines the memory strategy for Scene loading and instantiation, with particular attention to activation phases. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Activation phases is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with failure rollback is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for activation phases; distinguish required behavior from illustrative implementation detail.
- Keep failure rollback behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SceneLoadingAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SceneLoadingAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SceneLoadingAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.08 and activation phases.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.08 Scene loading and instantiation

Failure handling: failure rollback

This page defines the failure handling for Scene loading and instantiation, with particular attention to failure rollback. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Failure rollback is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with staging is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for failure rollback; distinguish required behavior from illustrative implementation detail.
- Keep staging behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Scene", "SceneLoadingAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.08 and failure rollback.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.08 Scene loading and instantiation

Diagnostics: staging

This page defines the diagnostics for Scene loading and instantiation, with particular attention to staging. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Staging is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with reference resolution is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for staging; distinguish required behavior from illustrative implementation detail.
- Keep reference resolution behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SceneLoadingAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SceneLoadingAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SceneLoadingAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.08 and staging.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.08 Scene loading and instantiation

Implementation sequence: reference resolution

This page defines the implementation sequence for Scene loading and instantiation, with particular attention to reference resolution. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Reference resolution is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with activation phases is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for reference resolution; distinguish required behavior from illustrative implementation detail.
- Keep activation phases behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Scene", "SceneLoadingAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.08 and reference resolution.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.08 Scene loading and instantiation

Verification: activation phases

This page defines the verification for Scene loading and instantiation, with particular attention to activation phases. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Activation phases is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with failure rollback is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for activation phases; distinguish required behavior from illustrative implementation detail.
- Keep failure rollback behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SceneLoadingAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SceneLoadingAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SceneLoadingAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.08 and activation phases.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.09 Spatial data structures

Purpose and boundary: BVH

This page defines the purpose and boundary for Spatial data structures, with particular attention to BVH. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Bvh is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with loose octree is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for BVH; distinguish required behavior from illustrative implementation detail.
- Keep loose octree behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SpatialDataStructuresService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SpatialDataStructuresConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SpatialDataStructuresState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.09 and BVH.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.09 Spatial data structures

Architecture: loose octree

This page defines the architecture for Spatial data structures, with particular attention to loose octree. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Loose octree is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with grids is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for loose octree; distinguish required behavior from illustrative implementation detail.
- Keep grids behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Spatial", "SpatialDataStructures");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.09 and loose octree.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.09 Spatial data structures

Data model: grids

This page defines the data model for Spatial data structures, with particular attention to grids. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Grids is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with update costs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for grids; distinguish required behavior from illustrative implementation detail.
- Keep update costs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SpatialDataStructuresService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SpatialDataStructuresConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SpatialDataStructuresState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.09 and grids.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.09 Spatial data structures

Public API: update costs

This page defines the public api for Spatial data structures, with particular attention to update costs. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Update costs is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with query APIs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for update costs; distinguish required behavior from illustrative implementation detail.
- Keep query APIs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Spatial", "SpatialDataStructures");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.09 and update costs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.09 Spatial data structures

Ownership and lifetime: query APIs

This page defines the ownership and lifetime for Spatial data structures, with particular attention to query APIs. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Query apis is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with BVH is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for query APIs; distinguish required behavior from illustrative implementation detail.
- Keep BVH behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SpatialDataStructuresService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SpatialDataStructuresConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SpatialDataStructuresState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.09 and query APIs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.09 Spatial data structures

Threading model: BVH

This page defines the threading model for Spatial data structures, with particular attention to BVH. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Bvh is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with loose octree is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for BVH; distinguish required behavior from illustrative implementation detail.
- Keep loose octree behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Spatial", "SpatialDataStructures");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.09 and BVH.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.09 Spatial data structures

Memory strategy: loose octree

This page defines the memory strategy for Spatial data structures, with particular attention to loose octree. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Loose octree is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with grids is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for loose octree; distinguish required behavior from illustrative implementation detail.
- Keep grids behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SpatialDataStructuresService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SpatialDataStructuresConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SpatialDataStructuresState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.09 and loose octree.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.09 Spatial data structures

Failure handling: grids

This page defines the failure handling for Spatial data structures, with particular attention to grids. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Grids is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with update costs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for grids; distinguish required behavior from illustrative implementation detail.
- Keep update costs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Spatial", "SpatialDataStructures");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.09 and grids.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.09 Spatial data structures

Diagnostics: update costs

This page defines the diagnostics for Spatial data structures, with particular attention to update costs. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Update costs is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with query APIs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for update costs; distinguish required behavior from illustrative implementation detail.
- Keep query APIs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SpatialDataStructuresService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SpatialDataStructuresConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SpatialDataStructuresState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.09 and update costs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.09 Spatial data structures

Implementation sequence: query APIs

This page defines the implementation sequence for Spatial data structures, with particular attention to query APIs. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Query apis is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with BVH is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for query APIs; distinguish required behavior from illustrative implementation detail.
- Keep BVH behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Spatial", "SpatialDataStructures");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.09 and query APIs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.09 Spatial data structures

Verification: BVH

This page defines the verification for Spatial data structures, with particular attention to BVH. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Bvh is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with loose octree is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for BVH; distinguish required behavior from illustrative implementation detail.
- Keep loose octree behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SpatialDataStructuresService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SpatialDataStructuresConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SpatialDataStructuresState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.09 and BVH.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.09 Spatial data structures

Completion gate: loose octree

This page defines the completion gate for Spatial data structures, with particular attention to loose octree. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Loose octree is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with grids is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for loose octree; distinguish required behavior from illustrative implementation detail.
- Keep grids behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Spatial", "SpatialDataStructures");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.09 and loose octree.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.09 Spatial data structures

Purpose and boundary: grids

This page defines the purpose and boundary for Spatial data structures, with particular attention to grids. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Grids is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with update costs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for grids; distinguish required behavior from illustrative implementation detail.
- Keep update costs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SpatialDataStructuresService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SpatialDataStructuresConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SpatialDataStructuresState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.09 and grids.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.09 Spatial data structures

Architecture: update costs

This page defines the architecture for Spatial data structures, with particular attention to update costs. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Update costs is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with query APIs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for update costs; distinguish required behavior from illustrative implementation detail.
- Keep query APIs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Spatial", "SpatialDataStructures");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.09 and update costs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.10 World cell partitioning

Purpose and boundary: cell IDs

This page defines the purpose and boundary for World cell partitioning, with particular attention to cell IDs. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Cell ids is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with boundaries is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cell IDs; distinguish required behavior from illustrative implementation detail.
- Keep boundaries behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class WorldCellPartitioningService final
{
public:
    [[nodiscard]] Result<void> Initialise(const WorldCellPartitioningConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    WorldCellPartitioningState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.10 and cell IDs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.10 World cell partitioning

Architecture: boundaries

This page defines the architecture for World cell partitioning, with particular attention to boundaries. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Boundaries is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with persistent layer is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for boundaries; distinguish required behavior from illustrative implementation detail.
- Keep persistent layer behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("World", "WorldCellPartitioning");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.10 and boundaries.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.10 World cell partitioning

Data model: persistent layer

This page defines the data model for World cell partitioning, with particular attention to persistent layer. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Persistent layer is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cross-cell references is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for persistent layer; distinguish required behavior from illustrative implementation detail.
- Keep cross-cell references behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class WorldCellPartitioningService final
{
public:
    [[nodiscard]] Result<void> Initialise(const WorldCellPartitioningConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    WorldCellPartitioningState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.10 and persistent layer.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.10 World cell partitioning

Public API: cross-cell references

This page defines the public api for World cell partitioning, with particular attention to cross-cell references. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Cross-cell references is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cell IDs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cross-cell references; distinguish required behavior from illustrative implementation detail.
- Keep cell IDs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("World", "WorldCellPartitioning");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.10 and cross-cell references.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.10 World cell partitioning

Ownership and lifetime: cell IDs

This page defines the ownership and lifetime for World cell partitioning, with particular attention to cell IDs. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Cell ids is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with boundaries is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cell IDs; distinguish required behavior from illustrative implementation detail.
- Keep boundaries behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class WorldCellPartitioningService final
{
public:
    [[nodiscard]] Result<void> Initialise(const WorldCellPartitioningConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    WorldCellPartitioningState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.10 and cell IDs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.10 World cell partitioning

Threading model: boundaries

This page defines the threading model for World cell partitioning, with particular attention to boundaries. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Boundaries is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with persistent layer is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for boundaries; distinguish required behavior from illustrative implementation detail.
- Keep persistent layer behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("World", "WorldCellPartitioning");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.10 and boundaries.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.10 World cell partitioning

Memory strategy: persistent layer

This page defines the memory strategy for World cell partitioning, with particular attention to persistent layer. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Persistent layer is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cross-cell references is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for persistent layer; distinguish required behavior from illustrative implementation detail.
- Keep cross-cell references behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class WorldCellPartitioningService final
{
public:
    [[nodiscard]] Result<void> Initialise(const WorldCellPartitioningConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    WorldCellPartitioningState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.10 and persistent layer.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.10 World cell partitioning

Failure handling: cross-cell references

This page defines the failure handling for World cell partitioning, with particular attention to cross-cell references. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Cross-cell references is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cell IDs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cross-cell references; distinguish required behavior from illustrative implementation detail.
- Keep cell IDs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("World", "WorldCellPartitioning");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.10 and cross-cell references.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.10 World cell partitioning

Diagnostics: cell IDs

This page defines the diagnostics for World cell partitioning, with particular attention to cell IDs. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Cell ids is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with boundaries is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cell IDs; distinguish required behavior from illustrative implementation detail.
- Keep boundaries behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class WorldCellPartitioningService final
{
public:
    [[nodiscard]] Result<void> Initialise(const WorldCellPartitioningConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    WorldCellPartitioningState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.10 and cell IDs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.10 World cell partitioning

Implementation sequence: boundaries

This page defines the implementation sequence for World cell partitioning, with particular attention to boundaries. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Boundaries is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with persistent layer is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for boundaries; distinguish required behavior from illustrative implementation detail.
- Keep persistent layer behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("World", "WorldCellPartitioning");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.10 and boundaries.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.10 World cell partitioning

Verification: persistent layer

This page defines the verification for World cell partitioning, with particular attention to persistent layer. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Persistent layer is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cross-cell references is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for persistent layer; distinguish required behavior from illustrative implementation detail.
- Keep cross-cell references behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class WorldCellPartitioningService final
{
public:
    [[nodiscard]] Result<void> Initialise(const WorldCellPartitioningConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    WorldCellPartitioningState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.10 and persistent layer.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.10 World cell partitioning

Completion gate: cross-cell references

This page defines the completion gate for World cell partitioning, with particular attention to cross-cell references. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Cross-cell references is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cell IDs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cross-cell references; distinguish required behavior from illustrative implementation detail.
- Keep cell IDs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("World", "WorldCellPartitioning");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.10 and cross-cell references.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.10 World cell partitioning

Purpose and boundary: cell IDs

This page defines the purpose and boundary for World cell partitioning, with particular attention to cell IDs. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Cell ids is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with boundaries is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cell IDs; distinguish required behavior from illustrative implementation detail.
- Keep boundaries behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class WorldCellPartitioningService final
{
public:
    [[nodiscard]] Result<void> Initialise(const WorldCellPartitioningConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    WorldCellPartitioningState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.10 and cell IDs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.10 World cell partitioning

Architecture: boundaries

This page defines the architecture for World cell partitioning, with particular attention to boundaries. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Boundaries is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with persistent layer is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for boundaries; distinguish required behavior from illustrative implementation detail.
- Keep persistent layer behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("World", "WorldCellPartitioning");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.10 and boundaries.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.11 Asynchronous world streaming

Purpose and boundary: request graph

This page defines the purpose and boundary for Asynchronous world streaming, with particular attention to request graph. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Request graph is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with priorities is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for request graph; distinguish required behavior from illustrative implementation detail.
- Keep priorities behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AsynchronousWorldStreamingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AsynchronousWorldStreamingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AsynchronousWorldStreamingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.11 and request graph.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.11 Asynchronous world streaming

Architecture: priorities

This page defines the architecture for Asynchronous world streaming, with particular attention to priorities. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Priorities is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cancellation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for priorities; distinguish required behavior from illustrative implementation detail.
- Keep cancellation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Asynchronous", "AsynchronousWorldStreaming");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.11 and priorities.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.11 Asynchronous world streaming

Data model: cancellation

This page defines the data model for Asynchronous world streaming, with particular attention to cancellation. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Cancellation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with residency is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cancellation; distinguish required behavior from illustrative implementation detail.
- Keep residency behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AsynchronousWorldStreamingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AsynchronousWorldStreamingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AsynchronousWorldStreamingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.11 and cancellation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.11 Asynchronous world streaming

Public API: residency

This page defines the public api for Asynchronous world streaming, with particular attention to residency. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Residency is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with handoff to rendering/physics is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for residency; distinguish required behavior from illustrative implementation detail.
- Keep handoff to rendering/physics behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Asynchronous", "AsynchronousWorldStreaming");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.11 and residency.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.11 Asynchronous world streaming

Ownership and lifetime: handoff to rendering/physics

This page defines the ownership and lifetime for Asynchronous world streaming, with particular attention to handoff to rendering/physics. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Handoff to rendering/physics is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with request graph is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for handoff to rendering/physics; distinguish required behavior from illustrative implementation detail.
- Keep request graph behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AsynchronousWorldStreamingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AsynchronousWorldStreamingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AsynchronousWorldStreamingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.11 and handoff to rendering/physics.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.11 Asynchronous world streaming

Threading model: request graph

This page defines the threading model for Asynchronous world streaming, with particular attention to request graph. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Request graph is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with priorities is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for request graph; distinguish required behavior from illustrative implementation detail.
- Keep priorities behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Asynchronous", "AsynchronousWorldStreaming");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.11 and request graph.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.11 Asynchronous world streaming

Memory strategy: priorities

This page defines the memory strategy for Asynchronous world streaming, with particular attention to priorities. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Priorities is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cancellation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for priorities; distinguish required behavior from illustrative implementation detail.
- Keep cancellation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AsynchronousWorldStreamingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AsynchronousWorldStreamingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AsynchronousWorldStreamingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.11 and priorities.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.11 Asynchronous world streaming

Failure handling: cancellation

This page defines the failure handling for Asynchronous world streaming, with particular attention to cancellation. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Cancellation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with residency is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cancellation; distinguish required behavior from illustrative implementation detail.
- Keep residency behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Asynchronous", "AsynchronousWorldStreaming");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.11 and cancellation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.11 Asynchronous world streaming

Diagnostics: residency

This page defines the diagnostics for Asynchronous world streaming, with particular attention to residency. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Residency is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with handoff to rendering/physics is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for residency; distinguish required behavior from illustrative implementation detail.
- Keep handoff to rendering/physics behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AsynchronousWorldStreamingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AsynchronousWorldStreamingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AsynchronousWorldStreamingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.11 and residency.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.11 Asynchronous world streaming

Implementation sequence: handoff to rendering/physics

This page defines the implementation sequence for Asynchronous world streaming, with particular attention to handoff to rendering/physics. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Handoff to rendering/physics is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with request graph is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for handoff to rendering/physics; distinguish required behavior from illustrative implementation detail.
- Keep request graph behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Asynchronous", "AsynchronousWorldStreaming");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.11 and handoff to rendering/physics.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.11 Asynchronous world streaming

Verification: request graph

This page defines the verification for Asynchronous world streaming, with particular attention to request graph. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Request graph is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with priorities is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for request graph; distinguish required behavior from illustrative implementation detail.
- Keep priorities behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AsynchronousWorldStreamingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AsynchronousWorldStreamingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AsynchronousWorldStreamingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.11 and request graph.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.11 Asynchronous world streaming

Completion gate: priorities

This page defines the completion gate for Asynchronous world streaming, with particular attention to priorities. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Priorities is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cancellation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for priorities; distinguish required behavior from illustrative implementation detail.
- Keep cancellation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Asynchronous", "AsynchronousWorldStreaming");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.11 and priorities.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.11 Asynchronous world streaming

Purpose and boundary: cancellation

This page defines the purpose and boundary for Asynchronous world streaming, with particular attention to cancellation. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Cancellation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with residency is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cancellation; distinguish required behavior from illustrative implementation detail.
- Keep residency behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AsynchronousWorldStreamingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AsynchronousWorldStreamingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AsynchronousWorldStreamingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.11 and cancellation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.11 Asynchronous world streaming

Architecture: residency

This page defines the architecture for Asynchronous world streaming, with particular attention to residency. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Residency is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with handoff to rendering/physics is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for residency; distinguish required behavior from illustrative implementation detail.
- Keep handoff to rendering/physics behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Asynchronous", "AsynchronousWorldStreaming");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.11 and residency.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.11 Asynchronous world streaming

Data model: handoff to rendering/physics

This page defines the data model for Asynchronous world streaming, with particular attention to handoff to rendering/physics. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Handoff to rendering/physics is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with request graph is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for handoff to rendering/physics; distinguish required behavior from illustrative implementation detail.
- Keep request graph behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AsynchronousWorldStreamingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AsynchronousWorldStreamingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AsynchronousWorldStreamingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.11 and handoff to rendering/physics.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.11 Asynchronous world streaming

Public API: request graph

This page defines the public api for Asynchronous world streaming, with particular attention to request graph. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Request graph is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with priorities is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for request graph; distinguish required behavior from illustrative implementation detail.
- Keep priorities behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Asynchronous", "AsynchronousWorldStreaming");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.11 and request graph.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.11 Asynchronous world streaming

Ownership and lifetime: priorities

This page defines the ownership and lifetime for Asynchronous world streaming, with particular attention to priorities. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Priorities is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cancellation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for priorities; distinguish required behavior from illustrative implementation detail.
- Keep cancellation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AsynchronousWorldStreamingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AsynchronousWorldStreamingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AsynchronousWorldStreamingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.11 and priorities.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.11 Asynchronous world streaming

Threading model: cancellation

This page defines the threading model for Asynchronous world streaming, with particular attention to cancellation. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Cancellation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with residency is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cancellation; distinguish required behavior from illustrative implementation detail.
- Keep residency behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Asynchronous", "AsynchronousWorldStreaming");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.11 and cancellation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.12 World state persistence

Purpose and boundary: runtime state

This page defines the purpose and boundary for World state persistence, with particular attention to runtime state. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Runtime state is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with save regions is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for runtime state; distinguish required behavior from illustrative implementation detail.
- Keep save regions behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class WorldStatePersistenceService final
{
public:
    [[nodiscard]] Result<void> Initialise(const WorldStatePersistenceConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    WorldStatePersistenceState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.12 and runtime state.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.12 World state persistence

Architecture: save regions

This page defines the architecture for World state persistence, with particular attention to save regions. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Save regions is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with dirty tracking is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for save regions; distinguish required behavior from illustrative implementation detail.
- Keep dirty tracking behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("World", "WorldStatePersistence");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.12 and save regions.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.12 World state persistence

Data model: dirty tracking

This page defines the data model for World state persistence, with particular attention to dirty tracking. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Dirty tracking is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with versioning is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for dirty tracking; distinguish required behavior from illustrative implementation detail.
- Keep versioning behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class WorldStatePersistenceService final
{
public:
    [[nodiscard]] Result<void> Initialise(const WorldStatePersistenceConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    WorldStatePersistenceState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.12 and dirty tracking.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.12 World state persistence

Public API: versioning

This page defines the public api for World state persistence, with particular attention to versioning. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Versioning is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with runtime state is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for versioning; distinguish required behavior from illustrative implementation detail.
- Keep runtime state behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("World", "WorldStatePersistence");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.12 and versioning.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.12 World state persistence

Ownership and lifetime: runtime state

This page defines the ownership and lifetime for World state persistence, with particular attention to runtime state. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Runtime state is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with save regions is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for runtime state; distinguish required behavior from illustrative implementation detail.
- Keep save regions behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class WorldStatePersistenceService final
{
public:
    [[nodiscard]] Result<void> Initialise(const WorldStatePersistenceConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    WorldStatePersistenceState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.12 and runtime state.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.12 World state persistence

Threading model: save regions

This page defines the threading model for World state persistence, with particular attention to save regions. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Save regions is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with dirty tracking is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for save regions; distinguish required behavior from illustrative implementation detail.
- Keep dirty tracking behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("World", "WorldStatePersistence");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.12 and save regions.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.12 World state persistence

Memory strategy: dirty tracking

This page defines the memory strategy for World state persistence, with particular attention to dirty tracking. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Dirty tracking is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with versioning is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for dirty tracking; distinguish required behavior from illustrative implementation detail.
- Keep versioning behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class WorldStatePersistenceService final
{
public:
    [[nodiscard]] Result<void> Initialise(const WorldStatePersistenceConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    WorldStatePersistenceState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.12 and dirty tracking.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.12 World state persistence

Failure handling: versioning

This page defines the failure handling for World state persistence, with particular attention to versioning. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Versioning is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with runtime state is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for versioning; distinguish required behavior from illustrative implementation detail.
- Keep runtime state behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("World", "WorldStatePersistence");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.12 and versioning.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.12 World state persistence

Diagnostics: runtime state

This page defines the diagnostics for World state persistence, with particular attention to runtime state. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Runtime state is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with save regions is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for runtime state; distinguish required behavior from illustrative implementation detail.
- Keep save regions behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class WorldStatePersistenceService final
{
public:
    [[nodiscard]] Result<void> Initialise(const WorldStatePersistenceConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    WorldStatePersistenceState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.12 and runtime state.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.13 Editor world tooling contracts

Purpose and boundary: cell visibility

This page defines the purpose and boundary for Editor world tooling contracts, with particular attention to cell visibility. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Cell visibility is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with locking is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cell visibility; distinguish required behavior from illustrative implementation detail.
- Keep locking behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class EditorWorldToolingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const EditorWorldToolingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    EditorWorldToolingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.13 and cell visibility.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.13 Editor world tooling contracts

Architecture: locking

This page defines the architecture for Editor world tooling contracts, with particular attention to locking. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Locking is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with streaming preview is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for locking; distinguish required behavior from illustrative implementation detail.
- Keep streaming preview behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Editor", "EditorWorldTooling");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.13 and locking.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.13 Editor world tooling contracts

Data model: streaming preview

This page defines the data model for Editor world tooling contracts, with particular attention to streaming preview. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Streaming preview is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with large-world navigation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for streaming preview; distinguish required behavior from illustrative implementation detail.
- Keep large-world navigation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class EditorWorldToolingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const EditorWorldToolingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    EditorWorldToolingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.13 and streaming preview.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.13 Editor world tooling contracts

Public API: large-world navigation

This page defines the public api for Editor world tooling contracts, with particular attention to large-world navigation. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Large-world navigation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cell visibility is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for large-world navigation; distinguish required behavior from illustrative implementation detail.
- Keep cell visibility behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Editor", "EditorWorldTooling");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.13 and large-world navigation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.13 Editor world tooling contracts

Ownership and lifetime: cell visibility

This page defines the ownership and lifetime for Editor world tooling contracts, with particular attention to cell visibility. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Cell visibility is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with locking is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cell visibility; distinguish required behavior from illustrative implementation detail.
- Keep locking behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class EditorWorldToolingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const EditorWorldToolingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    EditorWorldToolingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.13 and cell visibility.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.13 Editor world tooling contracts

Threading model: locking

This page defines the threading model for Editor world tooling contracts, with particular attention to locking. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Locking is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with streaming preview is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for locking; distinguish required behavior from illustrative implementation detail.
- Keep streaming preview behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Editor", "EditorWorldTooling");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.13 and locking.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.13 Editor world tooling contracts

Memory strategy: streaming preview

This page defines the memory strategy for Editor world tooling contracts, with particular attention to streaming preview. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Streaming preview is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with large-world navigation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for streaming preview; distinguish required behavior from illustrative implementation detail.
- Keep large-world navigation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class EditorWorldToolingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const EditorWorldToolingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    EditorWorldToolingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.13 and streaming preview.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.14 Precision and streaming test programme

Purpose and boundary: extreme coordinates

This page defines the purpose and boundary for Precision and streaming test programme, with particular attention to extreme coordinates. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Extreme coordinates is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with seams is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for extreme coordinates; distinguish required behavior from illustrative implementation detail.
- Keep seams behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PrecisionAndStreamingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PrecisionAndStreamingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PrecisionAndStreamingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.14 and extreme coordinates.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.14 Precision and streaming test programme

Architecture: seams

This page defines the architecture for Precision and streaming test programme, with particular attention to seams. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Seams is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with teleports is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for seams; distinguish required behavior from illustrative implementation detail.
- Keep teleports behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Precision", "PrecisionAndStreaming");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.14 and seams.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.14 Precision and streaming test programme

Data model: teleports

This page defines the data model for Precision and streaming test programme, with particular attention to teleports. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Teleports is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with load storms is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for teleports; distinguish required behavior from illustrative implementation detail.
- Keep load storms behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PrecisionAndStreamingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PrecisionAndStreamingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PrecisionAndStreamingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.14 and teleports.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.14 Precision and streaming test programme

Public API: load storms

This page defines the public api for Precision and streaming test programme, with particular attention to load storms. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Load storms is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with determinism is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for load storms; distinguish required behavior from illustrative implementation detail.
- Keep determinism behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Precision", "PrecisionAndStreaming");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.14 and load storms.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.14 Precision and streaming test programme

Ownership and lifetime: determinism

This page defines the ownership and lifetime for Precision and streaming test programme, with particular attention to determinism. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Determinism is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with extreme coordinates is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for determinism; distinguish required behavior from illustrative implementation detail.
- Keep extreme coordinates behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PrecisionAndStreamingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PrecisionAndStreamingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PrecisionAndStreamingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.14 and determinism.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.14 Precision and streaming test programme

Threading model: extreme coordinates

This page defines the threading model for Precision and streaming test programme, with particular attention to extreme coordinates. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Extreme coordinates is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with seams is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for extreme coordinates; distinguish required behavior from illustrative implementation detail.
- Keep seams behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Precision", "PrecisionAndStreaming");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 03.14 and extreme coordinates.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

03.14 Precision and streaming test programme

Memory strategy: seams

This page defines the memory strategy for Precision and streaming test programme, with particular attention to seams. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Seams is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with teleports is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for seams; distinguish required behavior from illustrative implementation detail.
- Keep teleports behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class PrecisionAndStreamingService final
{
public:
    [[nodiscard]] Result<void> Initialise(const PrecisionAndStreamingConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    PrecisionAndStreamingState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 03.14 and seams.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.